

llama.cpp README

Source URL: <https://raw.githubusercontent.com/ggml-org/llama.cpp/master/README.md>

llama.cpp

Manifesto / ggml / ops

LLM inference in C/C++

Recent API changes

- **Changelog for libllama API**
- **Changelog for llama-server REST API**

Hot topics

- **Hugging Face cache migration:** models downloaded with -hf are now stored in the standard Hugging Face cache directory, enabling sharing with other HF tools.

guide : using the new WebUI of llama.cpp

guide : running gpt-oss with llama.cpp

- **[[FEEDBACK] Better packaging for llama.cpp to support downstream consumers**
■](<https://github.com/ggml-org/llama.cpp/discussions/15313>)

- Support for the gpt-oss model with native MXFP4 format has been added | PR | Collaboration with NVIDIA | Comment

Multimodal support arrived in llama-server: 12898 | documentation

- VS Code extension for FIM completions: <https://github.com/ggml-org/llama.vscode>

Vim/Neovim plugin for FIM completions: https://github.com/ggml-org/llama.vim

- Hugging Face Inference Endpoints now support GGUF out of the box!
<https://github.com/ggml-org/llama.cpp/discussions/9669>

Hugging Face GGUF editor: discussion | tool

- WebGPU support is now available in the browser, see a blog/demo introducing it here.

Quick start

Getting started with llama.cpp is straightforward. Here are several ways to install it on your machine:

- **Install llama.cpp using brew, nix, winget, or conda-forge**
- **Run with Docker - see our Docker documentation**
- **Download pre-built binaries from the releases page**
- **Build from source by cloning this repository - check out our build guide**

Once installed, you'll need a model to work with. Head to the Obtaining and quantizing models section to learn more.

Example command:

```
```sh
```

#### **Use a local model file**

```
llama-cli -m my_model.gguf
```

#### **Or download and run a model directly from Hugging Face**

```
llama-cli -hf ggml-org/gemma-3-1b-it-GGUF
```

#### **Launch OpenAI-compatible API server**

```
llama-server -hf ggml-org/gemma-3-1b-it-GGUF
```

```
```
```

Description

The main goal of llama.cpp is to enable LLM inference with minimal setup and state-of-the-art performance on a wide

range of hardware - locally and in the cloud.

- Plain C/C++ implementation without any dependencies

- Apple silicon is a first-class citizen - optimized via ARM NEON, Accelerate and Metal frameworks

- AVX, AVX2, AVX512 and AMX support for x86 architectures

- RVV, ZVfH, Zfh, Zicbop and Zihintpause support for RISC-V architectures

- 1.5-bit, 2-bit, 3-bit, 4-bit, 5-bit, 6-bit, and 8-bit integer quantization for faster inference and reduced memory use

- Custom CUDA kernels for running LLMs on NVIDIA GPUs (support for AMD GPUs via HIP and Moore Threads GPUs via MUSA)

- Vulkan and SYCL backend support

- CPU+GPU hybrid inference to partially accelerate models larger than the total VRAM capacity

The llama.cpp project is the main playground for developing new features for the ggml library.

Models

Typically finetunes of the base models below are supported as well.

Instructions for adding support for new models: HOWTO-add-model.md

Text-only

- [X] LLaMA ■

- [x] LLaMA 2 ■■

- [x] LLaMA 3 ■■■■
- [X] Mistral 7B
- [x] Mixtral MoE
- [x] DBRX
- [x] Jamba
- [X] Falcon
- [X] Chinese LLaMA / Alpaca and Chinese LLaMA-2 / Alpaca-2
- [X] Vigogne (French)
- [X] BERT
- [X] Koala
- [X] Baichuan 1 & 2 + derivations
- [X] Aquila 1 & 2
- [X] Starcoder models
- [X] Refact
- [X] MPT
- [X] Bloom
- [x] Yi models
- [X] StableLM models
- [x] Deepseek models
- [x] Qwen models
- [x] PLaMo-13B
- [x] Phi models
- [x] PhiMoE
- [x] GPT-2
- [x] Orion 14B
- [x] InternLM2
- [x] CodeShell
- [x] Gemma
- [x] Mamba

- [x] **Grok-1**
- [x] **Xverse**
- [x] **Command-R models**
- [x] **SEA-LION**
- [x] **GritLM-7B + GritLM-8x7B**
- [x] **OLMo**
- [x] **OLMo 2**
- [x] **OLMoE**
- [x] **Granite models**
- [x] **GPT-NeoX + Pythia**
- [x] **Snowflake-Arctic MoE**
- [x] **Smaug**
- [x] **Poro 34B**
- [x] **Bitnet b1.58 models**
- [x] **Flan T5**
- [x] **Open Elm models**
- [x] **ChatGLM3-6b + ChatGLM4-9b + GLMEdge-1.5b + GLMEdge-4b**
- [x] **GLM-4-0414**
- [x] **SmolLM**
- [x] **EXAONE-3.0-7.8B-Instruct**
- [x] **FalconMamba Models**
- [x] **Jais**
- [x] **Bielik-11B-v2.3**
- [x] **RWKV-7**
- [x] **RWKV-6**
- [x] **QRWKV-6**
- [x] **GigaChat-20B-A3B**
- [X] **Trillion-7B-preview**
- [x] **Ling models**

- [x] *LFM2 models*
- [x] *Hunyuan models*
- [x] *BailingMoeV2 (Ring/Ling 2.0) models*
- [x] *Mellum models*

Multimodal

- [x] *LLaVA 1.5 models, LLaVA 1.6 models*
- [x] *BakLLaVA*
- [x] *Obsidian*
- [x] *ShareGPT4V*
- [x] *MobileVLM 1.7B/3B models*
- [x] *Yi-VL*
- [x] *Mini CPM*
- [x] *Moondream*
- [x] *Bunny*
- [x] *GLM-EDGE*
- [x] *Qwen2-VL*
- [x] *LFM2-VL*

Bindings

- *Python: ddh0/easy-llama*
- *Python: abetlen/llama-cpp-python*
- *Go: go-skynet/go-llama.cpp*
- *Node.js: withcatai/node-llama-cpp*
- *JS/TS (llama.cpp server client): lgrammel/modelfusion*
- *JS/TS (Programmable Prompt Engine CLI): offline-ai/cli*
- *JavaScript/Wasm (works in browser): tangledgroup/llama-cpp-wasm*
- *Typescript/Wasm (nicer API, available on npm): ngxson/wllama*
- *Ruby: yoshoku/llama_cpp.rb*
- *Ruby: docusealco/rllama*
- *Rust (more features): edgenai/llama_cpp-rs*

- **Rust (nicer API):** *mdrokz/rust-llama.cpp*
- **Rust (more direct bindings):** *utilityai/llama-cpp-rs*
- **Rust (automated build from crates.io):** *ShelbyJenkins/llm_client*
- **C/.NET:** *SciSharp/LLamaSharp*
- **C/VB.NET (more features - community license):** *LM-Kit.NET*
- **Scala 3:** *donderom/llm4s*
- **Clojure:** *phronmophobic/llama.clj*
- **React Native:** *mybigday/llama.rn*
- **Java:** *kherud/java-llama.cpp*
- **Java:** *QuasarByte/llama-cpp-jna*
- **Zig:** *deins/llama.cpp.zig*
- **Flutter/Dart:** *netdur/llama_cpp_dart*
- **Flutter:** *xuegao-tzx/Fllama*
- **PHP (API bindings and features built on top of llama.cpp):** *distantmagic/resonance* (more info)
- **Guile Scheme:** *guile_llama_cpp*
- **Swift** *srgtuszy/llama-cpp-swift*
- **Swift** *ShenghaiWang/SwiftLlama*
- **Delphi Embarcadero/llama-cpp-delphi**
- **Go (no CGo needed):** *hybridgroup/yzma*
- **Android:** *llama.android*

UIs

(to have a project listed here, it should clearly state that it depends on llama.cpp)

- **AI Sublime Text plugin (MIT)**
- **BonzAI App (proprietary)**
- **cztoomsik/ava (MIT)**
- **Dot (GPL)**
- **eva (MIT)**
- **iohub/collama (Apache-2.0)**
- **janhq/jan (AGPL)**

- *johnbean393/Sidekick (MIT)*
- *KanTV (Apache-2.0)*
- *KodiBot (GPL)*
- *llama.vim (MIT)*
- *LARS (AGPL)*
- *Llama Assistant (GPL)*
- *LlamaLib (Apache-2.0)*
- *LLMFarm (MIT)*
- *LLMUnity (MIT)*
- *LMStudio (proprietary)*
- *LocalAI (MIT)*
- *LostRuins/koboldcpp (AGPL)*
- *MindMac (proprietary)*
- *MindWorkAI/AI-Studio (FSL-1.1-MIT)*
- *Mobile-Artificial-Intelligence/maid (MIT)*
- *Mozilla-Ocho/llamafire (Apache-2.0)*
- *nat/openplayground (MIT)*
- *nomic-ai/gpt4all (MIT)*
- *ollama/ollama (MIT)*
- *oobabooga/text-generation-webui (AGPL)*
- *PocketPal AI (MIT)*
- *psugihara/FreeChat (MIT)*
- *ptsochantaris/emeltai (MIT)*
- *pythops/tenere (AGPL)*
- *ramalama (MIT)*
- *semperai/amica (MIT)*
- *withcatai/catai (MIT)*
- *Autopen (GPL)*

Tools

- akx/ggify – download PyTorch models from Hugging Face Hub and convert them to GGML
- akx/ollama-dl – download models from the Ollama library to be used directly with llama.cpp
- crashr/gppm – launch llama.cpp instances utilizing NVIDIA Tesla P40 or P100 GPUs with reduced idle power consumption
- gpustack/gguf-parser - review/check the GGUF file and estimate the memory usage
- Styled Lines (proprietary licensed, async wrapper of inference part for game development in Unity3d with pre-built Mobile and Web platform wrappers and a model example)
- unslothai/unsloth – ■ exports/saves fine-tuned and trained models to GGUF (Apache-2.0)

Infrastructure

- Paddler - Open-source LLMOps platform for hosting and scaling AI in your own infrastructure

- GPUStack - Manage GPU clusters for running LLMs

- llama_cpp_canister - llama.cpp as a smart contract on the Internet Computer, using WebAssembly
- llama-swap - transparent proxy that adds automatic model switching with llama-server

- Kalavai - Crowdsourced end to end LLM deployment at any scale

- llmaz - ■■ Easy, advanced inference platform for large language models on Kubernetes.
- LLMKube - Kubernetes operator for llama.cpp with multi-GPU and Apple Silicon Metal

support"

Games

- Lucy's Labyrinth - A simple maze game where agents controlled by an AI model will try to trick you.

Supported backends

| Backend | Target devices |

| --- | --- |

| Metal | Apple Silicon |

| BLAS | All |

| BLIS | All |

| SYCL | Intel GPU |

| [OpenVINO [In Progress]](docs/backend/OPENVINO.md) | Intel CPUs, GPUs, and NPUs |

| MUSA | Moore Threads GPU |

| CUDA | Nvidia GPU |

| HIP | AMD GPU |

| ZenDNN | AMD CPU |

| **Vulkan** | **GPU** |

| **CANN** | **Ascend NPU** |

| **OpenCL** | **Adreno GPU** |

| **IBM zDNN** | **IBM Z & LinuxONE** |

| **WebGPU** | **All** |

| **RPC** | **All** |

| **[Hexagon [In Progress]](docs/backend/snapdragon/README.md)** | **Snapdragon** |

| **VirtGPU** | **VirtGPU APIR** |

Obtaining and quantizing models

The Hugging Face platform hosts a number of LLMs compatible with llama.cpp:

- Trending

- LLaMA

You can either manually download the GGUF file or directly use any llama.cpp-compatible models from Hugging Face or other model hosting sites, by using this CLI argument: -hf / [:quant]. For example:

```
``sh
```

```
llama-cli -hf ggml-org/gemma-3-1b-it-GGUF
```

```
...
```

By default, the CLI would download from Hugging Face, you can switch to other options with the environment variable MODEL_ENDPOINT. The MODEL_ENDPOINT must point to a Hugging Face compatible API endpoint.

After downloading a model, use the CLI tools to run it locally - see below.

llama.cpp requires the model to be stored in the GGUF file format. Models in other data formats can be converted to GGUF using the convert_*.py Python scripts in this repo.

The Hugging Face platform provides a variety of online tools for converting, quantizing and hosting models with llama.cpp:

- Use the GGUF-my-repo space to convert to GGUF format and quantize model weights to smaller sizes

- Use the GGUF-my-LoRA space to convert LoRA adapters to GGUF format (more info: <https://github.com/ggml-org/llama.cpp/discussions/10123>)

- Use the GGUF-editor space to edit GGUF meta data in the browser (more info: <https://github.com/ggml-org/llama.cpp/discussions/9268>)

- Use the Inference Endpoints to directly host llama.cpp in the cloud (more info: <https://github.com/ggml-org/llama.cpp/discussions/9669>)

To learn more about model quantization, read this documentation

llama-cli

A CLI tool for accessing and experimenting with most of llama.cpp's functionality.

-

Run in conversation mode

Models with a built-in chat template will automatically activate conversation mode. If this doesn't occur, you can manually enable it by adding `-cnv` and specifying a suitable chat template with `--chat-template NAME`

```
``bash
```

llama-cli -m model.gguf

> hi, who are you?

Hi there! I'm your helpful assistant! I'm an AI-powered chatbot designed to assist and provide information to users like you. I'm here to help answer your questions, provide guidance, and offer support on a wide range of topics. I'm a friendly and knowledgeable AI, and I'm always happy to help with anything you need. What's on your mind, and how can I assist you today?

> what is 1+1?

Easy peasy! The answer to 1+1 is... 2!

```
```
```

-

### ***Run in conversation mode with custom chat template***

```
``bash
```

***use the "chatml" template (use -h to see the list of supported templates)***

***llama-cli -m model.gguf -cnv --chat-template chatml***

***use a custom template***

***llama-cli -m model.gguf -cnv --in-prefix 'User: ' --reverse-prompt 'User:'***

```
```
```

-

Constrain the output with a custom grammar

```
``bash
```

`llama-cli -m model.gguf -n 256 --grammar-file grammars/json.gbnf -p 'Request: schedule a call at 8pm; Command:'`

{"appointmentTime": "8pm", "appointmentDetails": "schedule a a call"}

```
```
```

The `grammars/` folder contains a handful of sample grammars. To write your own, check out the GBNF Guide.

For authoring more complex JSON grammars, check out <https://grammar.intrinsiclabs.ai/>

***llama-server***

A lightweight, OpenAI API compatible, HTTP server for serving LLMs.

-

***Start a local HTTP server with default configuration on port 8080***

```
``bash
```

```
llama-server -m model.gguf --port 8080
```

***Basic web UI can be accessed via browser: <http://localhost:8080>***

***Chat completion endpoint: <http://localhost:8080/v1/chat/completions>***

```
```
```

-

Support multiple-users and parallel decoding

```
``bash
```

up to 4 concurrent requests, each with 4096 max context

```
llama-server -m model.gguf -c 16384 -np 4
```

```
```
```

-

***Enable speculative decoding***

```
``bash
```

***the draft.gguf model should be a small variant of the target model.gguf***

```
llama-server -m model.gguf -md draft.gguf
```

```
```
```

-

Serve an embedding model

```
``bash
```

use the /embedding endpoint

```
llama-server -m model.gguf --embedding --pooling cls -ub 8192
```

```
```
```

-

***Serve a reranking model***

```
``bash
```

*use the /reranking endpoint*

***llama-server -m model.gguf --reranking***

...

-

*Constrain all outputs with a grammar*

***```bash***

*custom grammar*

***llama-server -m model.gguf --grammar-file grammar.gbnf***

***JSON***

***llama-server -m model.gguf --grammar-file grammars/json.gbnf***

...

***llama-perplexity***

A tool for measuring the perplexity <sup>[^1]</sup> (and other quality metrics) of a model over a given text.

-

*Measure the perplexity over a text file*

***```bash***

***llama-perplexity -m model.gguf -f file.txt***

[1]15.2701,[2]5.4007,[3]5.3073,[4]6.2965,[5]5.8940,[6]5.6096,[7]5.7942,[8]4.9297, ...

***Final estimate: PPL = 5.4007 +/- 0.67339***

...

-

*Measure KL divergence*

***```bash***

***TODO***

...

***[^1]: <https://huggingface.co/docs/transformers/perplexity>***

***llama-bench***

Benchmark the performance of the inference for various parameters.

-

## ***Run default benchmark***

```
``bash
```

***llama-bench -m model.gguf***

### ***Output:***

```
| model | size | params | backend | threads | test | t/s |
| ----- | -----: | -----: | ----- | -----: | -----: | -----: |
| qwen2 1.5B Q4_0 | 885.97 MiB | 1.54 B | Metal,BLAS | 16 | pp512 | 5765.41 ± 20.55 |
| qwen2 1.5B Q4_0 | 885.97 MiB | 1.54 B | Metal,BLAS | 16 | tg128 | 197.71 ± 0.81 |
```

***build: 3e0ba0e60 (4229)***

```
```
```

llama-simple

A minimal example for implementing apps with llama.cpp. Useful for developers.

-

Basic text completion

```
``bash
```

llama-simple -m model.gguf

Hello my name is Kaitlyn and I am a 16 year old girl. I am a junior in high school and I am currently taking a class called "The Art of

```
```
```

## ***Contributing***

- ***Contributors can open PRs***

- ***Collaborators will be invited based on contributions***

- Maintainers can push to branches in the llama.cpp repo and merge PRs into the master branch

- ***Any help with managing issues, PRs and projects is very appreciated!***

- ***See good first issues for tasks suitable for first contributions***

- ***Read the CONTRIBUTING.md for more information***

- ***Make sure to read this: Inference at the edge***

- ***A bit of backstory for those who are interested: Changelog podcast***

## ***Other documentation***

- ***cli***

- *completion*
- *server*
- *GBNF grammars*

### ***Development documentation***

- *How to build*
- *Running on Docker*
- *Build on Android*
- *Multi-GPU usage*
- *Performance troubleshooting*
- *GGML tips & tricks*

### ***Seminal papers and background on the models***

If your issue is with model generation quality, then please at least scan the following links and papers to understand the limitations of LLaMA models. This is especially important when choosing an appropriate model size and appreciating both the significant and subtle differences between LLaMA models and ChatGPT:

- ***LLaMA:***
- ***Introducing LLaMA: A foundational, 65-billion-parameter large language model***
- ***LLaMA: Open and Efficient Foundation Language Models***
- ***GPT-3***
- ***Language Models are Few-Shot Learners***
- ***GPT-3.5 / InstructGPT / ChatGPT:***
- ***Aligning language models to follow instructions***
- ***Training language models to follow instructions with human feedback***

### ***XCFramework***

The XCFramework is a precompiled version of the library for iOS, visionOS, tvOS,

***and macOS. It can be used in Swift projects without the need to compile the library from source. For example:***

```
``swift
```

```
// swift-tools-version: 5.10
```

```
// The swift-tools-version declares the minimum version of Swift required to build this package.
```

```
import PackageDescription
```

```

let package = Package(
 name: "MyLlamaPackage",
 targets: [
 .executableTarget(
 name: "MyLlamaPackage",
 dependencies: [
 "LlamaFramework"
],
),
 .binaryTarget(
 name: "LlamaFramework",
 url: "https://github.com/ggml-org/llama.cpp/releases/download/b5046/llama-b5046-xcframework.zip",
 checksum:
 "c19be78b5f00d8d29a25da41042cb7afa094cbf6280a225abe614b03b20029ab"
)
]
)

```

The above example is using an intermediate build b5046 of the library. This can be modified to use a different version by changing the URL and checksum.

## Completions

Command-line completion is available for some environments.

### Bash Completion

```

```bash
$ build/bin/llama-cli --completion-bash > ~/.llama-completion.bash
$ source ~/.llama-completion.bash
```

```

Optionally this can be added to your `.bashrc` or `.bash_profile` to load it automatically. For example:

```

```console
$ echo "source ~/.llama-completion.bash" >> ~/.bashrc

```

...

Dependencies

- yhirose/cpp-httpplib - Single-header HTTP server, used by llama-server - MIT license
- stb-image - Single-header image format decoder, used by multimodal subsystem - Public domain
- nlohmann/json - Single-header JSON library, used by various tools/examples - MIT License
- miniaudio.h - Single-header audio format decoder, used by multimodal subsystem - Public domain
- subprocess.h - Single-header process launching solution for C and C++ - Public domain